

Sistema de Facturación Electrónica ARCA

pullstok-erp · Backend (Express + Prisma) + Frontend (React/Vite) · Mapa del flujo completo

La clave: la facturación tiene **DOS números distintos** que conviene no confundir. El interno (FAC-XXXX) es de trazabilidad del sistema y no tiene valor fiscal. El fiscal (0002-00000008) es el real, otorgado por ARCA.

1. Los dos números

Número	Qué es	Quién lo genera
FAC-0001	Referencia interna del sistema (trazabilidad, antes de ser fiscal)	Contador por organización (Counter)
0002-00000008	Número fiscal real (punto de venta 4 díg + comprobante 8 díg)	ARCA / ArcaSequence

La factura nace como FAC-XXXX (sin valor fiscal). Recién cuando se emite contra ARCA se le asigna el número fiscal + CAE.

2. La integración ARCA (capa SOAP) — `api/src/integrations/arca/`

Capa que habla con AFIP/ARCA por SOAP, inyectable y mockeable (por eso se prueba sin red).

Archivo	Qué hace
types.ts	Dominios: <code>ArcaEnvironment</code> (HOMOLOGACION/PRODUCCION), <code>ArcaAuthContext</code> , <code>CaeRequest/CaeResult</code> , interface <code>ArcaClient</code> , códigos de error.
soapClient.ts	Transporte SOAP: arma el envelope, POST con <code>SOAPAction</code> , timeout y 1 retry. Parse con fast-xml-parser.
traSigner.ts	Arma el TRA (Ticket de Requerimiento de Acceso) y lo firma con CMS/PKCS#7 (node-forge, SHA-1). Gotcha: fechas sin milisegundos y en hora Buenos Aires con skew de 2 min.
wsaaClient.ts	<code>authenticateWsaa()</code> : LoginCms → Ticket de Acceso (TA). Cacheado 12hs en memoria. OJO: <code>LoginCmsReturn</code> es XML plano, no base64 (bug ya arreglado).
wsfev1Client.ts	Corazón de emisión: <code>fecaeSolicitar</code> (pedir CAE), <code>feCompUltimoAutorizado</code> (último número), <code>feCompConsultar</code> .
padronClient.ts	<code>getPersona()</code> : padrón A4 (autocompletar clientes). Endpoint nuevo de ARCA, anida en <code>personaReturn.persona</code> .
arcaClient.ts	Junta WSAA + WSFEv1 en la interface <code>ArcaClient</code> .
mockArcaClient.ts	Versión de memoria para tests/e2e sin red.

Ambientes

Se resuelven según `ArcaSetting.environment` (default: HOMOLOGACION):

- HOMOLOGACION → `https://wswhomo.afip.gov.ar/wsfev1/service.asmx`
- PRODUCCION → `https://servicios1.afip.gov.ar/wsfev1/service.asmx`

3. El orquestador de emisión — `api/src/services/fiscalInvoiceService.ts`

Dirige la emisión fiscal.

- `emitirFiscalmente(invoiceId, client, ctx)` — emite una factura `ISSUED` interna aún sin CAE.
- `reintentarFiscalmente(invoiceId, client, ctx)` — reintentar desde `PENDING_CAE` reutilizando el mismo `cbteNro`.
- `emitirCore(...)` — el corazón:
 1. Decide Factura A vs B (`deriveReceptorFiscal`): CUIT válido → A, DNI/sin CUIT → B, CUIT inválido → bloqueado.
 2. Reserva el correlativo (`cbteNro`).
 3. Marca `PENDING_CAE` + guarda campos fiscales (transacción corta).
 4. Pide el CAE a ARCA (**fuera** de la transacción).
 5. Si ARCA dice "ya registrado" → consulta para recuperar el CAE.
 6. Si falla → queda `PENDING_CAE` con código de error y contador.
 7. Si va bien → `ISSUED` con CAE + vencimiento.

Estados de la factura

```
DRAFT —issue()→ ISSUED (interna, sin CAE)
      —emitirFiscalmente→ PENDING_CAE (reservando número)
      —FECAESolicitar→ ISSUED (con CAE)
      fallo → PENDING_CAE (reintento, mismo cbteNro)
```

4. El cálculo fiscal — `api/src/services/arcaCalc.ts`

- Decide A vs B (`normalizeCuit` , `isValidCuit` , `deriveReceptorFiscal`).
- Mapea la alícuota interna al ID de ARCA (`mapALicuotaId` : 21→5, 10.5→4, etc.).
- `calcArcaAmounts` calcula montos en **centavos enteros** (evita errores de redondeo).

5. Controllers y el gate

Archivo	Qué hace
<code>invoiceController.ts</code>	CRUD de facturas + transiciones: <code>issueInvoice</code> (DRAFT→ISSUED), <code>issueFiscal</code> , <code>retryFiscal</code> , <code>markAsPaid</code> , <code>cancelInvoice</code> .
<code>arcaSettingsController.ts</code>	GET/PUT de configuración ARCA (solo ADMIN) + <code>getArcaEnabled</code> .
<code>padronController.ts</code>	<code>getPadronByCuit</code> (autocompletar cliente).
<code>checkArcaEnabled.ts</code>	Gate: si la org no tiene ARCA configurado → <code>403 ARCA_NOT_AVAILABLE</code> . Valida <code>enabled</code> + <code>cuitEmisor</code> + <code>puntoVenta</code> + <code>certPath</code> + <code>keyPath</code> .

6. Multi-tenant (organización)

- El cliente extendido de Prisma **inyecta automáticamente** `organizationId` en todos los modelos de tenant (Invoice, Product, Customer...).
- `ArcaSetting` , `ArcaSequence` y `Counter` están **fuera** del scope automático: se acceden explícitamente por `organizationId` vía `basePrisma` (evita fugas entre organizaciones).
- Los **certificados nunca van en la BD**: solo `certPath` / `keyPath` en el VPS (`/var/www/pullstok/certs/{org}/ ... crt / .key`).

7. El frontend

Archivo	Qué hace
<code>views/Invoices.tsx</code>	Listado. Arma el <code>InvoicePdfData</code> (<code>buildExport</code>) y dispara <code>window.print()</code> .
<code>views/InvoiceDetail.tsx</code>	Detalle (solo lectura). <code>PrintInvoice</code> siempre montado; el botón Imprimir llama <code>window.print()</code> .
<code>components/molecules/PrintInvoice</code>	Comprobante imprimible. Recibe <code>InvoicePdfData</code> , montado <code>hidden print:block</code> . Partes: franja ORIGINAL → header (recuadro de letra + emisor/comprobante) → cliente → tabla de ítems (se estira con <code>flex:1</code>) → totales → footer ARCA (QR + CAE).
<code>hooks/useInvoices.ts</code>	Data-fetching (query + mutations que refrescan la lista).
<code>hooks/useBranding.ts</code>	Logo de la org (<code>app-branding</code>).
<code>hooks/useAfipQrImage.ts</code>	Genera el QR como PNG para el <code>PrintInvoice</code> (canvas).
<code>utils/exportToPDF.ts</code>	Tipo <code>InvoicePdfData</code> / <code>InvoicePdfIssuer</code> + <code>exportToPDF</code> legacy (jsPDF).
<code>utils/afipQr.ts</code>	Payload base64 + URL <code>https://www.afip.gob.ar/fe/qr/?p=...</code> (RG 4892/2020).
<code>OrganizationFiscalForm</code>	Form de datos fiscales del emisor (CUIT, condición IVA, dirección, Ingresos Brutos, Inicio de Actividades).
<code>index.css</code>	<code>@page { size: A4; margin: 12mm }</code> + <code>@media print</code> (oculta todo, muestra solo <code>.print-area</code>).

OJO: hay **dos motores de QR** — jsPDF (`drawAfipQr`) para el export y canvas (`generateAfipQrDataUrl`) para el `PrintInvoice` . Misma lógica, dos implementaciones.

8. Flujo completo de emisión

1. Front: `useConfirm` → `issueInvoice(id)` → PUT `/api/invoices/:id/issue`
2. Backend: marca ISSUED interna con número FAC-XXXX
3. Si ARCA configurado:
 - reserva cbteNro (→ PENDING_CAE)
 - pide CAE a ARCA
 - OK → ISSUED con CAE
 - falla → PENDING_CAE + `fiscalError` (botón reintentar)
4. Si NO hay ARCA → ISSUED interna SIN CAE (no fiscal)

9. Flujo de impresión

1. Click "Imprimir" (listado o detalle)
2. Se monta `<PrintInvoice {datos} />` (`hidden print:block`)
3. `window.print()` → diálogo del navegador
4. CSS `@media print` oculta el resto y muestra solo `.print-area`

```
5. Navegador imprime/exporta a PDF el comprobante
   (con QR + CAE si está emitida; si no → "no válido como factura ARCA")
```

10. Gotchas que conviene conocer

- **Carrera de correlativo:** `getLastInvoiceNumber` corre fuera de transacción. Si dos emiten a la vez, pueden obtener el mismo `cbteNro` y ARCA responde "ya registrado" → el código consulta el CAE... *que es el de la otra factura* (edge case real de concurrencia).
- **Condición IVA del receptor con DNI** hardcodeda a 5 (Consumidor Final) aunque por norma podría ser RI.
- El `issue` devuelve **200 con `fiscalError` adjunto** cuando la emisión fiscal falla (en vez de 500) — el front lo interpreta para mostrar toast distinto.
- El **padrón en homologación solo acepta los CUITs de prueba de AFIP**; los reales dan "No existe persona" hasta que ARCA habilite producción (ticket 46924530).

11. Bug conocido: LoginCms WSAA

El `LoginCmsReturn` de WSAA **no contiene base64** del TA — contiene el XML del TA en texto plano escapado dentro del SOAP. El parser lo des-escapaba a string plano. La estructura interna del `TicketAcceso` es `LoginTicketResponse` con `<header>` (source/destination/uniqueId/generationTime/expirationTime) y `<credentials>` (token/sign).